



Help Me to Understand this Commit! - A Vision for Contextualized Code Reviews

Michael Unterkalmsteiner
Blekinge Institute of Technology, Software Engineering
Research Lab
Karlskrona, Sweden
michael.unterkalmsteiner@bth.se

Ricardo Britto
Ericsson AB and Blekinge Institute of Technology,
Software Engineering Research Lab
Karlskrona, Sweden
ricardo.britto@ericsson.com

Deepika Badampudi
Blekinge Institute of Technology, Software Engineering
Research Lab
Karlskrona, Sweden
deepika.badampudi@bth.se

Nauman bin Ali
Blekinge Institute of Technology, Software Engineering
Research Lab
Karlskrona, Sweden
nauman.ali@bth.se

ABSTRACT

Background: Modern Code Review (MCR) is a key component for delivering high-quality software and sharing knowledge among developers. Effective reviews require an in-depth understanding of the code and demand from the reviewers to contextualize the change from different perspectives. **Aim:** While there is a plethora of research on solutions that support developers to understand changed code, we have observed that many provide only narrow, specialized insights and very few aggregate information in a meaningful manner. Therefore, we aim to provide a vision of improving code understanding in MCR. **Method:** We classified 53 research papers suggesting proposals to improve MCR code understanding. We use this classification, the needs expressed by code reviewers from previous research, and the information we have *not* found in the literature for extrapolation. **Results:** We identified four major types of support systems and suggest an environment for contextualized code reviews. Furthermore, we illustrate with a set of scenarios how such an environment would improve the effectiveness of code reviews. **Conclusions:** Current research focuses mostly on providing narrow support for developers. We outline a vision for how MCR can be improved by using context and reducing the cognitive load on developers. We hope our vision can foster future advancements in development environments.

CCS CONCEPTS

• **Software and its engineering** → **Software verification and validation**; **Collaboration in software development**;

KEYWORDS

Modern code reviews, code understanding, decision-making, support systems

ACM Reference Format:

Michael Unterkalmsteiner, Deepika Badampudi, Ricardo Britto, and Nauman bin Ali. 2024. Help Me to Understand this Commit! - A Vision for Contextualized Code Reviews. In *2024 First IDE Workshop (IDE '24)*, April 20, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3643796.3648447>

1 INTRODUCTION

Modern Code Review (MCR) contributes to the code quality of software products [5] and knowledge dissemination in development organizations [6]. The practice of MCR was born out of the need to keep up with the ever-increasing speed of development and release cycles [1], replacing formal code inspections that were introduced by Fagan in the 1970s [14] and which have seen a peak in research in the late 1990s [22]. The MCR process relies on support that facilitates code distribution, feedback collection, and deciding whether to accept or reject a code change. The technological stack co-evolved with the MCR process comprises distributed version control systems (e.g., Git) and code collaboration tools (e.g., Gerrit and GitHub). The integration of these technologies has led to efficient management of the MCR process. To review code effectively, reviewers need to fully *understand* the code changes with all their implications. The current technology stack, including source code repositories, management systems, and integrated development environments, does not sufficiently support this. The most common approach to reviewing a change is to compare old and new code without any additional information beyond the description the author may or may not have provided.

Studies investigating code reviews in practice [1, 11, 28, 38] show that one of the main challenges experienced by code reviewers is access to the necessary information to make effective review decisions. These studies indicate that the situation has not considerably changed during the past ten years. In a recent study from 2022, Söderberg et al. [38] identified a misalignment between the *unit of analysis* that the review tools provide and the *unit of attention* that a code reviewer utilizes to make a decision. With a unit of analysis, they refer to the files automatically compiled by the review tool (such as Gerrit or GitHub) based on the changed code. On the other hand, the unit of attention is the information a reviewer needs to make an informed decision. A key finding of their study is that,



This work licensed under Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

IDE '24, April 20, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0580-9/24/04.

<https://doi.org/10.1145/3643796.3648447>

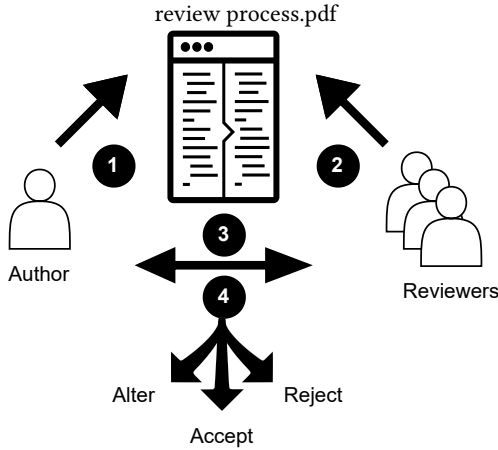


Figure 1: The code review process

while a review starts with the unit of analysis, reviewers often need to go beyond the affected files to understand the overall code change. In a recent survey [2], practitioners rated *understanding code under review* as the top research priority among existing MCR proposals.

In this paper, we provide an overview of the existing proposals to help reviewers understand a code change, i.e., proposals expanding the unit of analysis to approximate the needed unit of attention. We have grouped these proposals into four categories: refactoring, change impact, code reorganization, and data integration. However, few, if any, of these ideas have been transferred to the technology stack that is used in industrial practice today.

We argue that a fundamental problem of the identified solution proposals is their specialization on a single aspect of code understanding. They expand the unit of attention into one particular dimension only, for example, by decomposing significant changes into logical units. Other information needs are not satisfied by that particular solution proposal, and the unit of analysis and unit of attention remain misaligned. Without taking away the contributions that have been made in the past, we deem it necessary to rethink development environments for code reviews to increase their usefulness in practice. We envision a code collaboration environment whose main feature is providing context-dependent information, derived and synthesized from various artifacts and sources throughout the software development life-cycle and delivered on time.

2 BACKGROUND

Generally, the modern code review process (Figure 1) comprises the following four steps. The code author prepares a code change (Step 1), typically together with a textual description of the purpose of the change. One or more reviewers (the selection of those is out of scope for our discussion) review the change. The change can include completely new or deleted or changed files, typically processed by a tool that visualizes and highlights the changes (Step 2). The reviewers provide written feedback on the change, possibly referring to concrete lines of code or providing a general assessment of the change. Change authors can respond to this feedback (Step

3). The outcome of the discussion is a decision on whether to alter the code and return to Step 1, accept or reject the change (Step 4).

In this paper, we focus on Step 2, i.e., the information the reviewer(s) need to understand a change, evaluate it with respect to criteria that depend on the context of the review, and formulate actionable feedback for the change author.

2.1 Related work

Code reviewers' struggle with understanding code changes is not a new problem and has also been identified in traditional code inspections [10]. Bacchelli and Bird [1] reported in 2013, in one of the first studies of MCR, that understanding the reasons for a change is the most complex and time-consuming aspect of the review. Reviewers read the change description, run the code, or even reach out to authors to understand the code. Furthermore, tool support is limited to providing visual differences in the code, syntax highlighting, and commenting. A key factor in providing quick and valuable feedback to the change author is a priori knowledge of the change context [1].

Dong et al. [9] found in their survey, which covered scientific and gray literature between 2006 and 2020, that understanding code context is still the most challenging aspect for code reviewers. Baum and Schneider [4] called in 2016 for a new generation of code review tools that trade in the flexibility of the supported review process with support for code change understanding for the reviewer.

We argue that the trade-off posed by Baum and Schneider is not correct since the flexibility to support different review processes *and* the adaptive fulfillment of reviewers' information needs should be a core feature of a modern code collaboration/review environment. Our view is also supported by the results of Pascarella et al. [34], who studied the information needs of code reviewers. They conclude that the reviewers' information needs are on different conceptual levels and belong to different aspects of the code under review. Hence, the flexibility of the code review environment in terms of being able to satisfy information needs is essential.

To conclude, while context has been discussed many times (e.g., [1, 9, 11, 34]) as being essential for improving code change understanding, the idea to provide contextualized supporting information on-demand and tailored to the current review information needs has, to the best of our knowledge, not yet been proposed. This paper fills this gap.

3 RESEARCH METHODOLOGY

Our starting point was the following research question: *What aspects of code understanding for reviews are implemented in support systems?* To answer this question, we capitalized the results from a recent mapping study on MCR [2]. We focused on 23 primary studies categorized as solution proposals to improve the understanding of the code under review. In March 2023, we conducted one iteration of forward snowball sampling [40] on the start set, resulting in 351 papers. We identified 30 additional relevant papers. We read the full text of these 53 papers and extracted the investigated strategies to improve code understanding. The first author reviewed these strategies and identified four fundamental categories: refactoring, change impact, code reorganization, and data integration. Definitions of these categories are provided in Table 1. The remaining

Table 1: Code understanding categories used in review support systems

Category	Definition	Papers
Code reorganization	Code changes are reorganized (decomposed, reordered, regrouped) to allow for an easier understanding.	26
Data integration	The changed code is augmented with information, possibly in the form of visualizations, that provides additional context for the change. This information can but does not necessarily have to originate from the source code.	11
Refactoring	The changed code is augmented with information about refactorings such that reviewers understand better the nature of the change.	8
Change impact	Code changes are analyzed to understand how they impact related code (static analysis) or program behavior (dynamic analysis).	8

authors reviewed the categorization. The data set containing the sampled papers and the categorization are available online [39].

Validity threats. We performed only a single iteration of forward snowball sampling, which was, however, based on a starting set (23 papers) derived from a recent and rigorously conducted mapping study [2]. Another threat is that only the first author selected and categorized the studies from the snowball sample (30 papers), which may lead to the omission and misclassification of some studies. However, we reduced this threat by involving multiple researchers (study authors) in data extraction and developing the categories based on the 23 papers from the start-set. These stable categories and a shared understanding of the selection criteria helped to reduce the above threat. While we acknowledge these threats, we consider that our approach was sufficient to take a broader overview of research, reflect on research trends and practitioners' needs (see Söderberg et al. [38]), and sufficiently demonstrate a research gap.

4 CODE REVIEW CONTEXTS

As part of our vision of the future of code reviews, we introduce the idea of *code review contexts*¹. Murphy [33] has argued that the lack of context in software engineering tools limits developers' effectiveness. Therefore, we envision a code review environment that provides reviewers with the required information about the code change based on the review context.

The context can be global or local. Global means that the context is determined by the overall purpose of the review (e.g., identify security vulnerabilities) or the expertise of the reviewer (knowledge of the to-be-reviewed code). Local means that the context is determined by the file, class, method, or line, which is the current focus of the reviewer (unit of analysis). Depending on the global and local context, the reviewer has different information needs (unit of attention) that are satisfied by the envisioned code review environment, which we call Code Review Contextualizer (CoReCo) henceforth. Using the lens of code review contexts, we reviewed the existing literature to identify the support systems that help reviewers understand code changes. This review aims to illustrate that many information needs can already be satisfied but must be *integrated* into an environment that provides on-demand, contextualized support.

In the remainder of this section, we provide an overview of solutions that have been proposed to improve code understanding in reviews (Section 4.1), our vision of a next-generation code review environment, CoReCo, (Section 4.2), and application scenarios that illustrate the potential of CoReCo (Section 4.3).

4.1 What we found

Table 1 describes the four code understanding categories we identified for review support systems. Systems with code reorganization are the most frequent proposals, with 26 studies.

Code reorganization. Solutions are proposed to decompose composite change sets into different partitions based on (i) reasons for modifications, such as bug fixes and features, (ii) regrouping based on atomic changes addressing a single issue and related changes, and (iii) identifying essential parts of the commit. For example, SmartCommit [36], a graph-partitioning-based interactive approach, is proposed to decompose tangled changesets automatically. An interactive approach called DiffSearch [8] returns a set of changes matching a particular query. The queries are related to reasons for changing code, including code improvements and cleanup, functionality changes, bug fixes, and uses of a new API. Another approach is to help reviewers understand the salient class where the main change happens and triggers changes in the other classes. Huang et al. [19] propose a solution to characterize a class's salience based on several code and commit features.

Data integration. Providing a structural representation of the change can help reviewers understand the code to be reviewed. For example, Fregnan et al. [16] propose a visualization approach that displays classes and methods in review changes as nodes in a graph. UTANGO [26] is another tool that decomposes different groups with different concerns within the changed code and its surrounding code. UTANGO visualizes additional contextual embeddings for code changes that integrate program dependencies, changes, contexts, and cloned code.

Refactoring. Similar to the code reorganization category, the papers in this category reorganize the code with a focus on providing refactoring details. An example of such a solution is Refactorinsight [24], a tool that auto-folds and provides a refactoring description, which in turn helps reviewers to focus on changes related to bug fixes and new features. Solutions are also proposed to identify specific refactorings, such as removing code clones [7].

Change impact. This category includes solutions that provide behavioral summaries describing the impact of changes and enable a better understanding of the code to be reviewed [30]. Another example is a solution to provide relational information on the classes/methods/modules impacted by the change [32].

Synthesis. The categories described in this section answer our initial research question, i.e., what aspects of code understanding for reviews are implemented in support systems? The proposals integrating data from different sources to provide context for the reviewing decision are the most interesting ones concerning aggregating information meaningfully, thereby reducing the misalignment between a unit of analysis and the unit of attention. Our proposal

¹Encyclopædia Britannica defines to *contextualize* as "to think about or provide information about the situation in which something happens" <https://www.britannica.com/dictionary/contextualize>

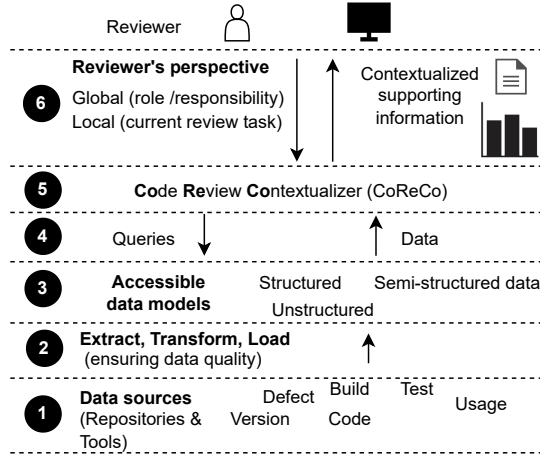


Figure 2: An overview of CoReCo's components

aims to develop this idea further, explicitly with the goal of *dynamic review contextualization based on the reviewers' needs*.

4.2 The Code Review Contextualizer (CoReCo)

We envision a future where code reviews remain critical, even in a world where Large Language Models (LLMs) co-write most code. While generative tools help developers to accelerate production and explore alternatives [3], humans remain essential to ensuring high-quality code through reviews.

Human code reviewers must consider numerous context factors when reviewing code. The relationship between a code change and a product's architecture, security and safety constraints, performance, privacy, interoperability, maintainability (and many more) requirements may influence the reviewing strategy. An additional critical aspect of efficient code reviews is the order in which to approach them. A linear review may not be the most effective strategy, as critical changes in a commit can be located in the middle of a file. Managing these factors requires experience and is often challenging without tool support. CoReCo is our vision of a code review environment that supports developers in making more efficient and effective review decisions as follows.

(1) CoReCo starts from the unit of analysis, i.e., the changed files, and analyzes its content, and, for example, change [31] and defect [41] history, frequency of execution in operation (e.g., through log mining techniques [13]), and impact on the remaining system [23]. By analyzing the content of changes and commit messages, CoReCo identifies (e.g., through topic modeling [37]) relevant documentation from requirements specifications and informal communication.

(2) CoReCo creates a profile of the unit of analysis with aspects (historically defect-prone code, mission-critical code, refactored code, etc.) that are likely important for the review. This profile, together with linked documentation, helps the reviewer to decide on what to focus on during the review.

(3) Reviewer experience is characterized based on past reviews and used to estimate the alignment to the expertise requirements of the unit of analysis. Reviewing activities can be captured in

real-time through data collection of interactions with CoReCo and biometrical data (e.g., detection of task-unrelated thoughts with webcams [20]). This allows CoReCo to track, in the long term, to what extent reviewer expertise and activities affect product quality.

(4) The interaction of the reviewer with CoReCo is primarily unidirectional, i.e., the system provides information consumed by the reviewer. However, we also foresee an interactive system (similar to Diffsearch [8], which returns information based on a query related to the code). Recent advancements in using LLMs for solving real-world problems, such as code explanation [25] and question-answering [35], are promising. Research on the potential of LLMs indicates their capability to comprehend code syntax rules but also their failure to comprehend (yet) dynamic program behavior [27].

Figure 2 provides an overview of CoReCo's logical elements. CoReCo will sit on top of a system providing access to data collected throughout the software development life-cycle, from managing requirements, project planning, development, debugging environments and testing to product deployment and product usage (1). The data collection mechanism (2) will cope with diverse data sources and perform quality control. One key deterrent to using data analytics is poor data quality [15]. To make the data usable, data models must be developed (3) that allow clients, such as CoReCo, to access data on demand (4). Inspiration for such a data model can come from the work by Martínez-Fernández et al. [29] who propose an ontology for software engineering data integration. CoReCo prepares queries and receives data based on the review context (5). Such context intertwines the reviews' global and local perspectives. The global perspective is predominately determined by the reviewers' role, responsibility, and expertise. They set the initial context of the review and determine the initial parameters for CoReCo to adapt the queries and displayed data to the reviewers' information needs. During the review, CoReCo determines the context, i.e., the currently viewed code block, either by keyboard and mouse interaction or by tracking the reviewers' eye movement (or a combination of these two approaches). We expect that the work on code writing assistants, such as Copilot², can be valuable here as they are also currently challenged to determine useful context [3]. Finally, the information synthesized by CoReCo is displayed to the reviewers (6) when they need it. This can be a combination of a dashboard presenting information before the review is conducted, parameterized by the reviewers' perspective, and pop-up information screens that appear when relevant during the review.

We envision CoReCo to support all the code understanding categories identified in Section 4.1, through the abstraction of data collection and extraction that renders the information accessible for upstream analysis tasks. We also foresee that this has the potential to boost the research in the data integration category, allowing the enrichment of review context with relevant information that is not easily accessible without such an infrastructure.

4.3 Application Scenarios

To illustrate how we envision CoReCo being used in practice, we present a series of scenarios showing how our vision extends the work we reviewed (see Section 4.1).

²<https://github.blog/2021-06-29-introducing-github-copilot-ai-pair-programmer>

Security and deployment context. An architect is reviewing code that belongs to an extensive distributed software system in the financial domain. In this case, we envision the architect getting relevant prompts from CoReCo when the code concerns authentication, authorization, or accounting (AAA). CoReCo will inform the architect where the code under review will be deployed and be accessible. Such information about AAA requirements and deployment plans will be beneficial for a reviewer to decide if the code has a security vulnerability. For example, if accounting is not a requirement, perhaps user management is not required as the code is part of a component that is deployed and accessible from only within the trusted network. CoReCo will process artifacts like the source code, the security policy, and deployment configurations.

High-risk code. A developer is reviewing a pull request for a business-critical product. Failure to detect an issue in this module could lead to severe consequences since it is used frequently by most customers, as usage data shows. In the past, changes in this module have led to integration test failures, and impact analysis shows that files beyond the modified ones are affected. This information gives the reviewer indications that API changes in this module need to be carefully reviewed. Additional meta-data about the pull request, such as the familiarity of the author with the module based on past commits, and the time to the next planned release date, support the reviewer in deciding on what and why to comment.

Code change history. A developer is reviewing an unfamiliar pull request. The original code was authored several years ago and has seen four changes. Six months ago, the second change led to an intense discussion between the original author and the developer proposing the change. The second change was ultimately rejected. The third and fourth changes were minor and only addressed code style. The description of the change under review raises similar points as the code change from six months ago. While this information does not make the review task less challenging, the provided history on the controversial change provides context for the reviewer.

5 FUTURE PLANS

We foresee three research themes that, while intertwined, could be investigated independently if the ultimate goal (transfer CoReCo to practice) and assumptions (that need to be verified throughout the research) are shared and agreed upon.

Data collection, quality control, and accessibility. There is a need to design and implement infrastructure for collecting and validating software engineering data at scale from heterogeneous sources, which allows prototypical research solutions to be elevated to production environments [15]. The components 1-4 shown in Figure 2 represent this goal. We expect that the design of such infrastructure can be informed by similar initiatives in different domains. For example, DKPro [12] provides a collection of reusable Natural Language Processing (NLP) software, enabled by a type system that creates an abstraction layer on top of otherwise incompatible tools, data formats, and models. Similarly, the WEKA platform [18] integrates various machine learning and data mining tools. No such infrastructure exists yet for software engineering analytics.

Data integration, synthesis, and visualization. We know the information needs of code reviewers [34]. We envision how those

information needs can be satisfied to improve code understanding and enable decision-making through an integrated environment. It is still open to see which data is useful and how it can be synthesized to support such decisions. Furthermore, to the best of our knowledge, there is no mechanism to describe context in a way that allows one to define the scope and depth of information necessary to make a decision in code reviews. Work on development task context [17, 21] can serve as a starting point here.

Code review decision making. The aggregation of data to enrich the decision-making process in code reviews can, however, also make the problem of analysis paralysis more prominent. The more criteria reviewers have to weigh in their decision, the more complex the solution-finding process becomes. For example, let's assume a code change intends to improve execution efficiency. Static code analysis of the changed code indicates that code readability is also decreased and below an acceptable threshold. In addition, the code change is located in a feature that has high business value (faster execution will lead to more revenue) but has proven to be defect-prone in the past. What does the reviewer decide? It is unclear to what extent contextualization can alleviate information overload and how it should be implemented to be effective.

6 CONCLUSIONS

This paper argues for a new generation of code review environments that provide developers with contextualized information to perform reviews. We outline the components of the Code Review Contextualizer (CoReCo) and describe application scenarios that illustrate how this environment would be able to address the information needs of code reviewers. Finally, we outline three research directions that may lead, if followed, to realizing our vision. We hope this work can inspire further in development environments, supporting improvements in developers' experience potentially leading to code quality and development speed gains.

ACKNOWLEDGMENTS

This work has been supported by ELLIT; the Strategic Research Area within IT and Mobile Communications, funded by the Swedish Government. The work has also been supported by a research grant for the GIST (reference number 20220235) and SERT project from the Knowledge Foundation in Sweden.

REFERENCES

- [1] Alberto Bacchelli and Christian Bird. 2013. Expectations, Outcomes, and Challenges of Modern Code Review. In *Proceedings International Conference on Software Engineering (ICSE)*. IEEE, 712–721.
- [2] Deepika Badampudi, Michael Unterkalmsteiner, and Ricardo Britto. 2023. Modern Code Reviews-A Survey of Literature and Practice. *ACM Transactions on Software Engineering and Methodology* (2023).
- [3] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (April 2023), 78:85–78:111. <https://doi.org/10.1145/3586030>
- [4] Tobias Baum and Kurt Schneider. 2016. On the need for a new generation of code review tools. In *17th International Conference on Product-Focused Software Process Improvement (PROFES)*. Springer, 301–308.
- [5] Gabriele Bavota and Barbara Russo. 2015. Four eyes are better than two: On the impact of code reviews on software quality. In *International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 81–90.
- [6] Amiangshu Bosu, Jeffrey C Carver, Christian Bird, Jonathan Orbeck, and Christopher Chockley. 2016. Process aspects and social dynamics of contemporary

- code review: Insights from open source development and industrial practice at microsoft. *IEEE Transactions on Software Engineering* 43, 1 (2016), 56–75.
- [7] Zhiyuan Chen, Young-Woo Kwon, and Myoungkyu Song. 2018. Clone refactoring inspection by summarizing clone refactorings and detecting inconsistent changes during software evolution. *Journal of Software: Evolution and Process* 30, 10 (2018), e1951.
 - [8] Luca Di Grazia, Paul Bredl, and Michael Pradel. 2022. DiffSearch: A Scalable and Precise Search Engine for Code Changes. *IEEE Transactions on Software Engineering* (2022).
 - [9] Liming Dong, He Zhang, Lanxin Yang, Zhiluo Weng, Xin Yang, Xin Zhou, and Zifan Pan. 2021. Survey on Pains and Best Practices of Code Review. In *28th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 482–491.
 - [10] A Dunsmore, M Roper, and M Wood. 2000. The Role of Comprehension in Software Inspection. *Journal of Systems and Software* 52, 2 (June 2000), 121–129.
 - [11] Felipe Ebert, Fernando Castor, Nicole Novielli, and Alexander Serebrenik. 2021. An exploratory study on confusion in code reviews. *Empirical Software Engineering* 26, 1 (2021), 1–48.
 - [12] Richard Eckart de Castilho and Iryna Gurevych. 2014. A Broad-Coverage Collection of Portable NLP Components for Building Shareable Analysis Pipelines. In *Workshop on Open Infrastructures and Analysis Frameworks for HLT*. Association for Computational Linguistics and Dublin City University, Dublin, Ireland, 1–11. <https://doi.org/10.3115/v1/W14-5201>
 - [13] Diana El-Masri, Fabio Petrillo, Yann-Gaël Guéhéneuc, Abdelwahab Hamou-Lhadj, and Anas Bouziane. 2020. A Systematic Literature Review on Automated Log Abstraction Techniques. *Information and Software Technology* 122 (June 2020), 106276. <https://doi.org/10.1016/j.infsof.2020.106276>
 - [14] M. E. Fagan. 1976. Design and code inspections to reduce errors in program development. *IBM Systems Journal* 15, 3 (1976), 182–211.
 - [15] Iris Figalist, Christoph Elsner, Jan Bosch, and Helena Holmström Olsson. 2022. Breaking the Vicious Circle: A Case Study on Why AI for Software Analytics and Business Intelligence Does Not Take off in Practice. *Journal of Systems and Software* 184 (Feb. 2022), 111135. <https://doi.org/10.1016/j.jss.2021.111135>
 - [16] Enrico Fregnan, Josua Fröhlich, Davide Spadini, and Alberto Bacchelli. 2023. Graph-based visualization of merge requests for code review. *Journal of Systems and Software* 195 (2023), 111506.
 - [17] Marko Gasparic, Gail C Murphy, and Francesco Ricci. 2017. A context model for IDE-based recommendation systems. *Journal of Systems and Software* 128 (2017), 200–219.
 - [18] Mark Hall, Eibe Frank, Geoffrey Holmes, Bernhard Pfahringer, Peter Reutemann, and Ian H Witten. 2009. The WEKA data mining software: an update. *ACM SIGKDD explorations newsletter* 11, 1 (2009), 10–18.
 - [19] Yuan Huang, Nan Jia, Xiangping Chen, Kai Hong, and Zibin Zheng. 2020. Code review knowledge perception: Fusing multi-features for salient-class location. *IEEE Transactions on Software Engineering* 48, 5 (2020), 1463–1479.
 - [20] Stephen Hutt, Aaron Wong, Alexandra Papoutsaki, Ryan S. Baker, Joshua I. Gold, and Caitlin Mills. 2023. Webcam-Based Eye Tracking to Detect Mind Wandering and Comprehension Errors. *Behavior Research Methods* (Jan. 2023). <https://doi.org/10.3758/s13428-022-02040-x>
 - [21] Mik Kersten and Gail C Murphy. 2006. Using task context to improve programmer productivity. In *Proceedings of the 14th ACM SIGSOFT international symposium on Foundations of software engineering*. 1–11.
 - [22] Sami Kollanus and Jussi Koskinen. 2009. Survey of software inspection research. *The Open Software Engineering Journal* 3, 1 (2009).
 - [23] Maria Kretsou, Elvira-Maria Arvanitou, Apostolos Ampatzoglou, Ignatios Deligiannis, and Vassilis C. Gerogiannis. 2021. Change Impact Analysis: A Systematic Mapping Study. *Journal of Systems and Software* 174 (April 2021), 110892. <https://doi.org/10.1016/j.jss.2020.110892>
 - [24] Zarina Kurbatova, Vladimir Kovalenko, Ioana Savu, Bob Brockbernd, Dan Andreescu, Matei Anton, Roman Venediktov, Elena Tikhomirova, and Timofey Bryksin. 2021. Refactorinsight: Enhancing ide representation of changes in git with refactorings information. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1276–1280.
 - [25] Juho Leinonen, Paul Denny, Stephen MacNeil, Sami Sarsa, Seth Bernstein, Joanne Kim, Andrew Tran, and Arto Hellas. 2023. Comparing code explanations created by students and large language models. *arXiv preprint arXiv:2304.03938* (2023).
 - [26] Yi Li, Shaohua Wang, and Tien N Nguyen. 2022. UTANGO: untangling commits with context-aware, graph-based, code change clustering learning model. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 221–232.
 - [27] Wei Ma, Shangqing Liu, Wenhan Wang, Qiang Hu, Ye Liu, Cen Zhang, Liming Nie, and Yang Liu. 2023. The Scope of ChatGPT in Software Engineering: A Thorough Investigation. <https://doi.org/10.48550/arXiv.2305.12138> arXiv:cs/2305.12138
 - [28] Laura MacLeod, Michaela Greiler, Margaret-Anne Storey, Christian Bird, and Jacek Czerwinka. 2017. Code reviewing in the trenches: Challenges and best practices. *IEEE Software* 35, 4 (2017), 34–42.
 - [29] Silverio Martínez-Fernández, Petar Jovanovic, Xavier Franch, and Andreas Jedlitschka. 2018. Towards Automated Data Integration in Software Analytics. In *Proceedings of the International Workshop on Real-Time Business Intelligence and Analytics (BIRTE '18)*. Association for Computing Machinery, New York, NY, USA, 1–5. <https://doi.org/10.1145/3242153.3242159>
 - [30] Massimiliano Menarini, Yan Yan, and William G Griswold. 2017. Semantics-assisted code review: An efficient tool chain and a user study. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 554–565.
 - [31] Ran Mo, Shaozhi Wei, Qiong Feng, and Zengyang Li. 2022. An Exploratory Study of Bug Prediction at the Method Level. *Information and Software Technology* 144 (April 2022), 106794. <https://doi.org/10.1016/j.infsof.2021.106794>
 - [32] Amit Kumar Mondal, Chanchal K Roy, Kevin A Schneider, Banani Roy, and Sristy Sumana Nath. 2021. Semantic slicing of architectural change commits: Towards semantic design review. In *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 1–6.
 - [33] Gail C Murphy. 2019. Beyond integrated development environments: adding context to software development. In *2019 IEEE/ACM 41st international conference on software engineering: new ideas and emerging results (ICSE-NIER)*. IEEE, 73–76.
 - [34] Luca Pascarella, Davide Spadini, Fabio Palomba, Magiel Bruntink, and Alberto Bacchelli. 2018. Information needs in contemporary code review. *Proceedings of the ACM on Human-Computer Interaction* 2, CSCW (2018), 1–27.
 - [35] Fabio Petroni, Tim Rocktäschel, Sebastian Riedel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, and Alexander Miller. 2019. Language Models as Knowledge Bases?. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics.
 - [36] Bo Shen, Wei Zhang, Christian Kästner, Haiyan Zhao, Zhao Wei, Guangtai Liang, and Zhi Jin. 2021. SmartCommit: a graph-based interactive assistant for activity-oriented commits. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 379–390.
 - [37] Camila Costa Silva, Matthias Galster, and Fabian Gilson. 2021. Topic Modeling in Software Engineering Research. *Empirical Software Engineering* 26, 6 (Sept. 2021), 120. <https://doi.org/10.1007/s10664-021-10026-0>
 - [38] Emma Söderberg, Luke Church, Jürgen Börstler, Diederick Niehorster, and Christofer Rydénfält. 2022. Understanding the Experience of Code Review: Misalignments, Attention, and Units of Analysis. In *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering 2022*. 170–179.
 - [39] Michael Unterkalmsteiner, Deepika Badampudi, Ricardo Britto, and Nauman bin Ali. 2023. Help Me to Understand this Commit! - A Vision for Contextualized Code Reviews: data set. <https://doi.org/10.5281/zenodo.7889591>
 - [40] Claes Wohlin. 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th international conference on evaluation and assessment in software engineering*. 1–10.
 - [41] Qiao Yu, Shujuan Jiang, Junyan Qian, Lili Bo, Li Jiang, and Gongjie Zhang. 2020. Process Metrics for Software Defect Prediction in Object-Oriented Programs. *IET Software* 14, 3 (2020), 283–292. <https://doi.org/10.1049/iet-sen.2018.5439>